

Ejercicio 1

En el código que se le ha proporcionado se encuentra desarrollado la definición de un evento. Y asociado a este, se definió un método receptor que mediante el uso de un delegado realiza una acción cuando el evento se ejecuta (o se dispara). El evento está definido para ejecutarse cuando finaliza la transacción de un pago en línea.

Tenemos por un lado una pasarela de pago, que es quién procesa las transacciones de un nuevo pago realizado. Una vez que la transacción asociada a ese pago finaliza, es cuando se dispara el evento mencionado.

Por otro, tenemos un gestor de email que se encarga de enviar notificaciones a los usuarios. En este caso particular, al finalizar una transacción de pago se envía un mail notificando que fue procesado correctamente.

Para ello es que necesitamos que el emisor (pasarela de pago) del evento de fin de la transacción este vinculado a un método del receptor (gestor de email) para que se envíe al usuario la notificación mencionada cuando el pago se haya realizado. Si analiza todo el código, podrá observar en detalle cómo se ha definido este evento y como se realiza el envío del mail al finalizar la transacción.

Cuando lea y analice el código, notará que dentro del Main falta algo. Presta especial atención a los comentarios y verá que algunos de ellos dicen al comienzo la palabra **TO DO**. Esto quiere decir, que deberás completar allí el código que falta para que todo este desarrollo funcione de manera correcta. En dicho comentario se explica lo que se debe desarrollar en esa línea de código faltante.

Copia el siguiente código para comenzar el ejercicio.

```
using System;

namespace Coding.Exercise
{
    public class Exercise
    {
        //Clase de transacción con sus atributos
        public class Transaccion
        {
            public int identificador { get; set; }
            public string fechaTransaccion { get; set; }
        }

        //EventArgs personalizado para un evento de transacción
        public class TransaccionEventArgs : EventArgs
        {
            public Transaccion transaccion { get; set; }
        }

        //Clase para la gestión de pagos (Pasarela)
        public class PasarelaDePago
        {
            //Evento de fin de transacción
            public event EventHandler<TransaccionEventArgs> TransaccionFinalizada;

            //Método que dispara el evento de fin de transacción
            protected virtual void EnTransaccionFinalizada(Transaccion transaccion_) {
                //Disparo el evento de fin de transacción, mediante una llamada al mismo, solo si es distinto de null
                if(TransaccionFinalizada != null)
                    TransaccionFinalizada(this, new TransaccionEventArgs() { transaccion = transaccion_ });
            }

            //Método que procesa el pago mediante una transacción
            public void Pago(Transaccion transaccion_) {
                //Procesando transacción de pago
                Thread.Sleep(2000);

                //Pago aprobado y fin de transacción
                EnTransaccionFinalizada(transaccion_);
            }
        }
    }
}
```

Ejercicio 2

A partir de la solución del ejercicio anterior, ahora se pide asociar otro método (receptor) para el evento (emisor) de fin de transacción. La función de este método será notificar al usuario sobre la emisión de la factura correspondiente al pago realizado. Para esto, debes crear una nueva clase (**GestorDeFacturacion**) e implementar dentro de ella el método correspondiente al evento de fin de transacción. Dentro del método, y utilizando los datos de la transacción que recibimos como argumento, se debe notificar sobre la emisión de la factura imprimiendo en consola el siguiente mensaje:

"La factura correspondiente a la transacción 71645331 fue emitida con fecha 30/06/2010."

Luego, dentro del Main, deberás declarar la nueva clase de facturación y asociar el nuevo método receptor al evento de su emisor. Al igual que en el ejercicio anterior, seguir las

CFGs Desarrollo de Aplicaciones Multiplataforma
Módulo: Desarrollo de Interfaces

indicaciones de los comentarios con el prefijo **TO DO** para desarrollar esta nueva solución.

Mira y usa lo que necesites del siguiente código como punto de partida:

```
using System;

namespace Coding.Exercise
{
    public class Exercise
    {
        //Clase de transacción con sus atributos
        public class Transaccion
        {
            public int identificador { get; set; }
            public string fechaTransaccion { get; set; }
        }

        //EventArgs personalizado para un evento de transacción
        public class TransaccionEventArgs : EventArgs
        {
            public Transaccion transaccion { get; set; }
        }

        //Clase para la gestión de pagos (Pasarela)
        public class PasarelaDePago
        {
            //Evento de fin de transacción
            public event EventHandler<TransaccionEventArgs> TransaccionFinalizada;

            //Método que dispara el evento de fin de transacción
            protected virtual void EnTransaccionFinalizada(Transaccion transaccion_) {
                //Disparo el evento de fin de transacción, mediante una llamada al mismo, solo si es distinto de null
                if(TransaccionFinalizada != null)
                    TransaccionFinalizada(this, new TransaccionEventArgs() { transaccion = transaccion_ });
            }

            //Método que procesa el pago mediante una transacción
            public void Pago(Transaccion transaccion_) {
                //Procesando transacción de pago
                Thread.Sleep(2000);

                //Pago aprobado y fin de transacción
                EnTransaccionFinalizada(transaccion_);
            }
        }
    }
}
```